

33-data-visualization

33. Data Visualization

Level: □ Beginner → □ Intermediate **Jam:** 8 (4 minggu × 2 sesi) **Prasyarat:** JavaScript fundamentals, HTML/CSS, basic DOM manipulation **Output:** Interactive data dashboard — real-time charts, multiple chart types, D3.js custom viz

Tujuan Pembelajaran

Setelah modul ini, kamu bisa:

- Bedain chart types (line, bar, pie, scatter, area) dan kapan pake masing-masing
- Setup & konfigurasi Chart.js 4 — datasets, options, responsive, animations
- Bikin mixed charts, time series, zoom/pan, annotations, custom tooltips
- Export chart sebagai image
- Paham D3.js core concepts: select, data join, scale, axis, SVG
- Bikin bar chart & line chart dari scratch pake D3.js
- Transisi & animasi data di D3.js
- Integrasi real-time data via WebSocket + Chart.js
- Bikin dashboard layout pake CSS Grid — dark mode, filtering, date range
- Sinkronisasi multiple chart di dashboard

Materi

Sesi	Topik	File
1	Chart Basics — chart types, Chart.js setup, datasets & options, responsive & animations, tooltip & legend, multiple datasets	01-chart-basics.md
2	Chart Advanced — mixed charts, time series, zoom/pan, annotations, custom tooltips, plugins, export image	02-chart-advanced.md
3	D3.js Intro — select & data join, scale & axis, SVG basics, bar chart from scratch, line chart with axes, transition & animation	03-d3-intro.md
4	Dashboard Real-World — real-time data (WebSocket + Chart.js), CSS Grid dashboard layout, dark mode, filtering & date range, multiple chart sync	04-dashboard-realworld.md

Output Akhir Modul

Interactive Data Dashboard — Dashboard real-time dengan Chart.js (line, bar, mixed), D3.js custom viz, dark mode, filtering, date range picker, dan WebSocket data stream. Siap dipasang di aplikasi web mana pun.

AI Prompt Exercises

Sepanjang modul, latihan pake AI:

- “Generate Chart.js config for a line chart with 3 datasets and custom tooltip callback”
- “Convert this Chart.js bar chart to a horizontally stacked bar chart”

- “Explain D3 data join — enter, update, exit — with a simple example”
- “Write a D3.js bar chart from an array of objects with ordinal scale”
- “Help me debug this Chart.js chart — labels not showing on x-axis”
- “Generate a CSS Grid dashboard layout with 4 charts, sidebar, and header”
- “Compare Chart.js vs D3.js — when should I use which?”
- “Write a WebSocket client that feeds real-time data into Chart.js time series”

33.1 Chart Basics: Types, Chart.js Setup & Configuration

Chart Types Overview

Chart Type	Use Case	Contoh
Line	Tren sepanjang waktu	Revenue bulanan, temperature harian
Bar	Perbandingan antar kategori	Penjualan per produk, populasi per kota
Pie / Doughnut	Proporsi / persentase	Market share, budget allocation
Scatter	Korelasi 2 variabel	Height vs weight, price vs rating
Area	Volume sepanjang waktu, stacked comparison	Website traffic, resource usage
Radar	Multi-dimensi comparison	Skill matrix, product comparison
Bubble	3 variabel (x, y, ukuran)	GDP vs population vs area

Kapan Pake Chart Apa?

Data trend time-series? → Line chart

Bandingin kategori? → Bar chart (horizontal kalo kategori panjang)
 Persentase / pie of whole? → Pie (maks 5 slice) atau stacked bar
 Korelasi 2 numeric variable? → Scatter plot
 Distribusi data? → Histogram (bar chart dengan bin)
 Komposisi berubah waktu? → Stacked area chart

Chart.js Setup

CDN — langsung di HTML

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Chart.js Basics</title>
  <script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.1/dist/chart.umd.min.js"></script>
  <style>
    body { font-family: system-ui, sans-serif; max-width: 800px; margin: 0 auto; }
    canvas { max-width: 100%; }
  </style>
</head>
<body>
  <h1> Chart.js Demo</h1>
  <div style="width: 100%; max-width: 600px;">
    <canvas id="myChart"></canvas>
  </div>

  <script>
    const ctx = document.getElementById('myChart').getContext('2d');

    new Chart(ctx, {
      type: 'line',
      data: {
        labels: ['Jan', 'Feb', 'Mar', 'Apr', 'Mei', 'Jun'],
        datasets: [{
          label: 'Penjualan 2024',
          data: [12, 19, 15, 22, 28, 35],
          borderColor: '#3b82f6',
          backgroundColor: 'rgba(59, 130, 246, 0.1)',
          tension: 0.3,
          fill: true,
        }]
      },
      options: {
        responsive: true,
      }
    });
  </script>

```

```

        plugins: {
            title: { display: true, text: 'Penjualan Bulanan' }
        }
    });
</script>
</body>
</html>

```

NPM — untuk project modern

```
npm install chart.js
```

```

import { Chart, registerables } from 'chart.js';
Chart.register(...registerables);

const ctx = document.getElementById('myChart') as HTMLCanvasElement;
new Chart(ctx, {
    type: 'bar',
    data: { /* ... */ },
    options: { /* ... */ },
});

```

Datasets & Options

Dataset Properties

```

const dataset = {
    label: 'Revenue',
    data: [10, 20, 30],
    backgroundColor: 'rgba(59, 130, 246, 0.5)', // fill color
    borderColor: '#3b82f6', // line/border color
    borderWidth: 2,
    pointBackgroundColor: '#fff',
    pointBorderColor: '#3b82f6',
    pointRadius: 4,
    tension: 0.4, // curve smoothing (line)
    fill: true, // fill area under line
    hidden: false, // hide dataset (toggle legend)
};

```

Global Options

```

const options = {
    responsive: true, // auto resize on window change
    maintainAspectRatio: true, // maintain height/width ratio
};

```

```

interaction: {
  mode: 'index',           // nearest point, index, dataset, x, y
  intersect: false,
},
plugins: {
  legend: {
    display: true,
    position: 'top',       // top, bottom, left, right
    labels: { color: '#333' },
  },
  title: {
    display: true,
    text: 'Chart Title',
    font: { size: 18, weight: 'bold' },
  },
  tooltip: {
    enabled: true,
    mode: 'index',
    intersect: false,
  },
},
scales: {
  x: {
    title: { display: true, text: 'Bulan' },
    grid: { display: false },
  },
  y: {
    title: { display: true, text: 'Nilai' },
    beginAtZero: true,
    grid: { color: 'rgba(0,0,0,0.05)' },
  },
},
};

```

Responsive & Animations

```

const options = {
  responsive: true,
  maintainAspectRatio: false, // biar chart stretch sesuai container
  animation: {
    duration: 1000,           // ms
    easing: 'easeInOutQuart', // easeOutBounce, easeInOutCubic, etc.
    delay: (context) => {
      // stagger animation per bar/point
      return context.dataIndex * 100;
    }
  }
};

```

```

    },
  },
  transitions: {
    show: {
      animations: { x: { from: 0 }, y: { from: 0 } }
    },
    hide: {
      animations: { x: { to: 0 }, y: { to: 0 } }
    }
  }
};

```

Tooltip & Legend

Custom Tooltip Callback

```

const options = {
  plugins: {
    tooltip: {
      enabled: true,
      callbacks: {
        label: (context) => {
          const label = context.dataset.label || '';
          const value = context.parsed.y;
          return `${label}: Rp ${value.toLocaleString('id-ID')}`;
        },
        beforeLabel: (context) => {
          return `▢ ${context.dataIndex + 1}.`;
        },
        afterLabel: (context) => {
          const total = context.dataset.data.reduce((a, b) => a + b, 0);
          const pct = ((context.parsed.y / total) * 100).toFixed(1);
          return `(${pct}%)`;
        },
      },
    },
  },
  legend: {
    onClick: (event, legendItem, legend) => {
      // custom click - toggle visibility manual
      const index = legendItem.datasetIndex;
      const chart = legend.chart;
      const meta = chart.getDatasetMeta(index);
      meta.hidden = meta.hidden === null ? !chart.data.datasets[index].hidden : !meta.hidden;
      chart.update();
    },
    labels: {

```

```

        usePointStyle: true,           // circle/square instead of rect
        pointStyle: 'circle',
        padding: 20,
        font: { size: 13 },
      },
    },
  },
};

```

Multiple Datasets

```

<canvas id="multiChart" style="max-height: 400px;"></canvas>

<script>
  const ctx = document.getElementById('multiChart').getContext('2d');

  new Chart(ctx, {
    type: 'line',
    data: {
      labels: ['Q1', 'Q2', 'Q3', 'Q4'],
      datasets: [
        {
          label: 'Produk A',
          data: [15, 22, 18, 30],
          borderColor: '#3b82f6',
          backgroundColor: 'rgba(59, 130, 246, 0.1)',
          tension: 0.3,
        },
        {
          label: 'Produk B',
          data: [10, 18, 25, 28],
          borderColor: '#ef4444',
          backgroundColor: 'rgba(239, 68, 68, 0.1)',
          tension: 0.3,
        },
        {
          label: 'Produk C',
          data: [8, 12, 20, 22],
          borderColor: '#22c55e',
          backgroundColor: 'rgba(34, 197, 94, 0.1)',
          tension: 0.3,
        },
      ]
    },
    options: {

```



```

        responsive: true,
        interaction: { mode: 'index', intersect: false },
        plugins: {
            title: { display: true, text: 'Perbandingan Produk per Kwartal', font: {
            },
            scales: {
                y: { beginAtZero: true },
            },
        },
    });
</script>

```

Bar Chart & Pie Chart

Bar Chart

```

new Chart(ctx, {
    type: 'bar',
    data: {
        labels: ['Jakarta', 'Bandung', 'Surabaya', 'Medan', 'Yogyakarta'],
        datasets: [{
            label: 'Populasi (juta)',
            data: [10.56, 2.44, 2.97, 2.11, 0.42],
            backgroundColor: [
                '#3b82f6', '#ef4444', '#22c55e', '#f59e0b', '#8b5cf6'
            ],
        }],
    },
    options: {
        indexAxis: 'y', // horizontal bar
        scales: { x: { beginAtZero: true } },
    },
});

```

Pie / Doughnut

```

new Chart(ctx, {
    type: 'doughnut', // atau 'pie'
    data: {
        labels: ['Desktop', 'Mobile', 'Tablet'],
        datasets: [{
            data: [55, 35, 10],
            backgroundColor: ['#3b82f6', '#22c55e', '#f59e0b'],
            borderWidth: 2,
        }],
    },
});

```

```

    },
    options: {
      plugins: {
        legend: { position: 'bottom' },
        tooltip: {
          callbacks: {
            label: (ctx) => {
              const total = ctx.dataset.data.reduce((a, b) => a + b, 0);
              const pct = ((ctx.parsed / total) * 100).toFixed(1);
              return `${ctx.label}: ${pct}%`;
            },
          },
        },
      },
    },
  },
});

```

Scatter Plot

```

new Chart(ctx, {
  type: 'scatter',
  data: {
    datasets: [{
      label: 'Harga vs Rating',
      data: [
        { x: 15000, y: 4.5 },
        { x: 25000, y: 4.2 },
        { x: 5000, y: 3.8 },
        { x: 35000, y: 4.7 },
        { x: 12000, y: 3.5 },
      ],
      backgroundColor: '#3b82f6',
      pointRadius: 6,
    }]
  },
  options: {
    scales: {
      x: { title: { display: true, text: 'Harga (Rp)' } },
      y: { title: { display: true, text: 'Rating' }, min: 0, max: 5 },
    },
  },
});

```

Latihan

1. **Multi-Chart Dashboard:** Bikin 1 HTML dengan 3 chart (line, bar, pie) dalam satu halaman. Data: penjualan bulanan (Jan-Dec) untuk 2 produk. Line chart tampilkan tren, bar chart bandingin per bulan, pie chart tampilkan total per produk. Semua responsive.
2. **Custom Tooltip Currency:** Bikin chart bar untuk data pengeluaran bulanan (makan: 2.5jt, transport: 1.2jt, hiburan: 800rb, tagihan: 1.5jt, lainnya: 600rb). Tooltip harus format Rupiah (Rp 2.500.000) dan tampilkan persentase dari total. Tambah legend horizontal bar.
3. **Animated Scatter Plot:** Bikin scatter plot dengan 30 data point random (x: 0-100, y: 0-100). Color points berdasarkan cluster (3 warna random). Tambah animasi stagger — tiap point muncul dengan delay 50ms. Gunakan `animation.delay` callback.
4. **Horizontal Stacked Bar:** Data: penjualan per kwartal (Q1-Q4) untuk 3 produk. Bikin horizontal stacked bar chart (`indexAxis: 'y'`). Setiap bar adalah 1 kwartal, stacked oleh 3 produk. Tambah tooltip yang show semua produk di kwartal itu.

33.2 Chart Advanced: Mixed Charts, Time Series, Plugins & Export

Mixed Charts

Chart.js support multiple chart types dalam 1 canvas pake mixed type.

```
<canvas id="mixedChart" style="max-height: 400px;"></canvas>
```

```
<script>
new Chart(document.getElementById('mixedChart'), {
  type: 'bar', // default type
  data: {
    labels: ['Jan', 'Feb', 'Mar', 'Apr', 'Mei', 'Jun'],
    datasets: [
      {
        label: 'Revenue',
        type: 'bar',
        data: [30, 45, 38, 52, 48, 62],
        backgroundColor: 'rgba(59, 130, 246, 0.6)',
        order: 2,
      },
    ],
  },
});
```

```

    {
      label: 'Target',
      type: 'line',
      data: [40, 40, 40, 50, 50, 50],
      borderColor: '#ef4444',
      borderWidth: 3,
      pointRadius: 5,
      pointBackgroundColor: '#ef4444',
      tension: 0,
      order: 1,
    },
    {
      label: 'Profit Margin (%)',
      type: 'line',
      yAxisID: 'y1',
      data: [22, 18, 25, 30, 28, 33],
      borderColor: '#22c55e',
      borderDash: [5, 5],
      pointRadius: 4,
      order: 1,
    },
  ],
},
options: {
  responsive: true,
  plugins: {
    title: { display: true, text: 'Revenue vs Target vs Profit Margin', font
    tooltip: { mode: 'index', intersect: false },
  },
  scales: {
    y: {
      position: 'left',
      title: { display: true, text: 'Revenue (jt)' },
      beginAtZero: true,
    },
    y1: {
      position: 'right',
      title: { display: true, text: 'Margin (%)' },
      grid: { drawOnChartArea: false },
      min: 0,
      max: 50,
    },
  },
},
});
</script>

```

Time Series

Chart.js pake adapter date library buat time series.

```
npm install chart.js chartjs-adapter-date-fns
```

```
# atau CDN:
```

```
# https://cdn.jsdelivr.net/npm/chartjs-adapter-date-fns@3/dist/chartjs-adapter-date-fns.min.js
```

```
<script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.1/dist/chart.umd.min.js">
```

```
<script src="https://cdn.jsdelivr.net/npm/chartjs-adapter-date-fns@3/dist/chartjs-adapter-date-fns.min.js">
```

```
<canvas id="timeChart"></canvas>
```

```
<script>
```

```
  // Generate hourly data for 7 days
```

```
  const now = new Date();
```

```
  const points = [];
```

```
  for (let i = 24 * 7; i >= 0; i--) {
```

```
    const date = new Date(now.getTime() - i * 60 * 60 * 1000);
```

```
    points.push({
```

```
      x: date,
```

```
      y: Math.floor(Math.random() * 50) + 20 + Math.sin(i / 12) * 15,
```

```
    });
```

```
  }
```

```
  new Chart(document.getElementById('timeChart'), {
```

```
    type: 'line',
```

```
    data: {
```

```
      datasets: [{
```

```
        label: 'CPU Usage (%)',
```

```
        data: points,
```

```
        borderColor: '#3b82f6',
```

```
        backgroundColor: 'rgba(59, 130, 246, 0.1)',
```

```
        fill: true,
```

```
        tension: 0.3,
```

```
        pointRadius: 0, // hide points for clean look
```

```
      }],
```

```
    },
```

```
    options: {
```

```
      responsive: true,
```

```
      plugins: {
```

```
        title: { display: true, text: 'CPU Usage – Last 7 Days', font: { size: 16 } },
```

```
        legend: { display: false },
```

```
      },
```

```
      scales: {
```

```
        x: {
```

```
          type: 'time',
```

```

        time: {
            unit: 'day',
            displayFormats: {
                hour: 'HH:mm',
                day: 'dd MMM',
                week: 'dd MMM',
            },
            tooltipFormat: 'dd MMM yyyy HH:mm',
        },
        title: { display: true, text: 'Waktu' },
    },
    y: {
        beginAtZero: true,
        max: 100,
        title: { display: true, text: 'CPU %' },
    },
},
});
</script>

```

Time Units

Unit	Contoh Format	Kapan Pake
millisecond	HH:mm:ss.SSS	Real-time high freq
second	HH:mm:ss	Per detik
minute	HH:mm	Per menit
hour	HH:mm / dd HH:mm	Intra-day
day	dd MMM	Harian
week	dd MMM	Mingguan
month	MMM yyyy	Bulanan
quarter	QQQ yyyy	Kwartal
year	yyyy	Tahunan

Zoom & Pan

Plugin chartjs-plugin-zoom.

npm install chartjs-plugin-zoom

```
<script src="https://cdn.jsdelivr.net/npm/chartjs-plugin-zoom@2/dist/chartjs-plu
```

```
<canvas id="zoomChart"></canvas>
```

```
<script>
```

```

new Chart(document.getElementById('zoomChart'), {
  type: 'line',
  data: {
    labels: Array.from({ length: 100 }, (_, i) => `Point ${i + 1}`),
    datasets: [{
      label: 'Noisy Signal',
      data: Array.from({ length: 100 }, () => Math.random() * 100),
      borderColor: '#8b5cf6',
      tension: 0.2,
      pointRadius: 1,
    }],
  },
  options: {
    responsive: true,
    plugins: {
      zoom: {
        pan: {
          enabled: true,
          mode: 'x',          // x, y, xy
        },
        zoom: {
          wheel: { enabled: true },
          pinch: { enabled: true },
          drag: {
            enabled: true,
            backgroundColor: 'rgba(139, 92, 246, 0.1)',
            borderColor: '#8b5cf6',
            borderWidth: 1,
          },
          mode: 'x',
        },
      },
    },
    scales: {
      x: { grid: { display: false } },
      y: { beginAtZero: true },
    },
  },
});
</script>

```

Annotations

Plugin `chartjs-plugin-annotation` buat nambah garis / label di chart.

```
npm install chartjs-plugin-annotation
```

```
new Chart(ctx, {
  type: 'line',
  data: {
    labels: ['Jan', 'Feb', 'Mar', 'Apr', 'Mei', 'Jun'],
    datasets: [{
      label: 'Revenue',
      data: [30, 45, 38, 52, 48, 62],
      borderColor: '#3b82f6',
      fill: false,
    }],
  },
  options: {
    plugins: {
      annotation: {
        annotations: {
          targetLine: {
            type: 'line',
            yMin: 50,
            yMax: 50,
            borderColor: '#ef4444',
            borderWidth: 2,
            borderDash: [6, 3],
            label: {
              display: true,
              content: 'Target 50jt',
              position: 'end',
            },
          },
        },
      },
      eventMarker: {
        type: 'point',
        xValue: 'Mar',
        yValue: 38,
        backgroundColor: '#f59e0b',
        radius: 8,
        label: {
          display: true,
          content: '□ Turun',
          position: 'bottom',
        },
      },
      regionHighlight: {
        type: 'box',
        xMin: 'Feb',
        xMax: 'Apr',
      },
    },
  },
});
```



```

        backgroundColor: 'rgba(239, 68, 68, 0.05)',
        borderWidth: 0,
      },
    },
  },
},
});

```

Custom Tooltips (HTML Tooltip)

Tooltip kustom render pake HTML — full control styling.

```

const options = {
  plugins: {
    tooltip: {
      enabled: false, // disable default
      external: (context) => {
        const { chart, tooltip } = context;
        let container = chart.canvas.parentNode.querySelector('.custom-tooltip')

        if (!container) {
          container = document.createElement('div');
          container.className = 'custom-tooltip';
          container.style.cssText = `
            position: absolute;
            background: #1e293b;
            color: white;
            padding: 10px 14px;
            border-radius: 8px;
            font-size: 13px;
            pointer-events: none;
            box-shadow: 0 4px 12px rgba(0,0,0,0.15);
            opacity: 0;
            transition: opacity 0.15s;
            z-index: 100;
          `;
          chart.canvas.parentNode.appendChild(container);
        }

        if (tooltip.opacity === 0) {
          container.style.opacity = '0';
          return;
        }

        // Build content

```

```

const title = tooltip.title?.[0] || '';
const body = tooltip.body?.map(b => b.lines.join(', ')).join('<br>') || '';

container.innerHTML = `
  <div style="font-weight:600;margin-bottom:4px;">${title}</div>
  <div>${body}</div>
`;

const pos = chart.canvas.getBoundingClientRect();
container.style.left = tooltip.caretX + 'px';
container.style.top = tooltip.caretY - container.offsetHeight - 10 + 'px';
container.style.opacity = '1';
  },
},
};

```

Chart.js Plugins

Register Plugin Global

```

import { Chart } from 'chart.js';

const backgroundPlugin = {
  id: 'customCanvasBackground',
  beforeDraw: (chart) => {
    const ctx = chart.ctx;
    ctx.save();
    ctx.fillStyle = '#f8f8f8';
    ctx.fillRect(0, 0, chart.width, chart.height);
    ctx.restore();
  },
};

Chart.register(backgroundPlugin);

```

Plugin buat Draw Line at Value

```

const horizontalLinePlugin = {
  id: 'horizontalLine',
  afterDraw: (chart, args, options) => {
    const { ctx, chartArea, scales } = chart;
    const yValue = options.value || 0;

    ctx.save();

```

```

        ctx.beginPath();
        ctx.moveTo(chartArea.left, scales.y.getPixelForValue(yValue));
        ctx.lineTo(chartArea.right, scales.y.getPixelForValue(yValue));
        ctx.strokeStyle = options.color || '#ef4444';
        ctx.lineWidth = options.lineWidth || 1;
        ctx.setLineDash([5, 3]);
        ctx.stroke();
        ctx.restore();
    },
};

// pake plugin
new Chart(ctx, {
    type: 'line',
    data: { /* ... */ },
    plugins: [horizontalLinePlugin],
    options: {
        plugins: {
            horizontalLine: {
                value: 50,
                color: '#ef4444',
                lineWidth: 2,
            },
        },
    },
});

```

Export Chart as Image

```

// Export ke PNG
function exportChart(chart, filename = 'chart.png') {
    const link = document.createElement('a');
    link.download = filename;
    link.href = chart.toBase64Image('image/png', 1); // type, quality
    link.click();
}

// Export ke JPEG
function exportJPEG(chart) {
    const link = document.createElement('a');
    link.download = 'chart.jpg';
    link.href = chart.toBase64Image('image/jpeg', 0.9);
    link.click();
}

```

```
// Export canvas langsung
function exportCanvas(canvasId) {
  const canvas = document.getElementById(canvasId);
  const image = canvas.toDataURL('image/png');
  const link = document.createElement('a');
  link.download = 'chart.png';
  link.href = image;
  link.click();
}

<button onclick="exportChart(myChart, 'revenue.png')">□ Download PNG</button>
<button onclick="exportJPEG(myChart)">□ Download JPEG</button>

<script>
  const myChart = new Chart(ctx, {
    type: 'bar',
    data: { /* ... */ },
    options: { /* ... */ },
  });
</script>
```

Latihan

1. **Mixed Chart dengan Dual Axis:** Data: pengunjung website per bulan (bar) dan conversion rate % (line dengan axis kanan). Bar chart warna biru, line chart merah dengan dashed style. Bulan Jan-Des. Tambah annotation garis target conversion rate 5%. Export button download PNG.
2. **Time Series dengan Zoom & Pan:** Generate 500 data point random timestamps (setiap 30 menit selama 10 hari). Tampilkan sebagai line chart time series. Aktifkan zoom drag, wheel zoom, dan pan. Tambah custom HTML tooltip yang nunjukin tanggal lengkap + nilai. Sembunyikan point biar rapi.
3. **Chart dengan Custom Plugin:** Bikin plugin Chart.js yang nge-draw grid vertikal setiap hari Senin (background color berbeda). Register plugin di chart time series. Plugin harus reusable — bisa di-set warna dan opacity lewat options.
4. **Annotation Dashboard:** Bikin chart line dengan 3 dataset (Actual, Budget, Forecast). Tambah annotations: (a) garis horizontal target tahunan, (b) box highlight Q3 region, (c) point marker di lowest actual value dengan label. Tambah label di tiap annotation. Export ke image setelah render.

33.3 D3.js Intro: Select, Data Join, Scale, SVG & Animation

Apa itu D3.js?

D3 (Data-Driven Documents) = library JavaScript buat manipulasi DOM berdasarkan data. Bedanya sama Chart.js: D3 kasi kontrol penuh atas visual — kamu bikin chart dari nol pake SVG.

Chart.js: high-level, instant chart, limited customization
D3.js: low-level, full control, steeper learning curve

Kapan Pake D3?

- Butuh custom visualization yang Chart.js gak bisa
- Butuh transisi & animasi kompleks
- Integrasi ketat dengan DOM / SVG
- Butuh reusable chart components
- Data visualization yang interaktif & unique

Setup

CDN

```
<script src="https://d3js.org/d3.v7.min.js"></script>
```

NPM

```
npm install d3
```

```
import * as d3 from 'd3';
```

Select & DOM Manipulation

```
// SELECT – mirip jQuery
d3.select('body')           // select first match
d3.selectAll('p')           // select all matches
d3.select('#chart')         // by ID
d3.selectAll('.bar')        // by class

// MANIPULASI
d3.select('h1')
  .style('color', '#3b82f6')
  .text('Hello D3!')
  .attr('class', 'title');
```

```
// APPEND
d3.select('body')
  .append('svg')
  .attr('width', 500)
  .attr('height', 300);
```

SVG Basics

Chart di D3 dibikin pake SVG (Scalable Vector Graphics).

```
<!-- SVG element -->
<svg width="500" height="300">
  <!-- Rectangle -->
  <rect x="10" y="20" width="40" height="100" fill="#3b82f6" />

  <!-- Circle -->
  <circle cx="100" cy="80" r="30" fill="#ef4444" />

  <!-- Line -->
  <line x1="150" y1="50" x2="250" y2="150" stroke="#22c55e" stroke-width="2" />

  <!-- Text -->
  <text x="300" y="100" font-size="14" fill="#333">Hello SVG</text>

  <!-- Group -->
  <g transform="translate(50, 50)">
    <rect x="0" y="0" width="50" height="50" fill="#f59e0b" />
    <text x="25" y="30" text-anchor="middle" fill="white">Grup</text>
  </g>

  <!-- Path (bentuk bebas) -->
  <path d="M 10 100 L 30 50 L 50 80 L 70 30 L 90 70"
        fill="none" stroke="#8b5cf6" stroke-width="2" />
</svg>
```

SVG Coordinate System

```
(0,0) → → → → → X →
      ↓
      ↓ Y increase ke BAWAH (kebalikan matematika)
      ↓
      Y
```

Data Join — Enter, Update, Exit

Core concept D3: bind data ke DOM elements.

```
const data = [10, 20, 30, 40, 50];

// JOIN data ke selection
const circles = d3.select('svg')
  .selectAll('circle')
  .data(data);

// ENTER — buat element baru untuk data yang belum ada
circles.enter()
  .append('circle')
  .attr('cx', (d, i) => 30 + i * 60)
  .attr('cy', 150)
  .attr('r', (d) => d)
  .attr('fill', '#3b82f6');

// UPDATE — update existing elements
circles
  .attr('fill', '#ef4444')
  .attr('r', (d) => d + 5);

// EXIT — remove elements yang gak ada datanya
circles.exit().remove();
```

General Update Pattern

```
function update(data) {
  const circles = svg.selectAll('circle').data(data);

  // ENTER
  const enter = circles.enter()
    .append('circle')
    .attr('cy', 150)
    .attr('fill', '#3b82f6');

  // MERGE — apply ke enter + update
  enter.merge(circles)
    .attr('cx', (d, i) => 30 + i * 60)
    .attr('r', (d) => d);

  // EXIT
  circles.exit().remove();
}
```

Scale

Scale = fungsi yang map data domain → pixel range.

```
// Linear Scale
const xScale = d3.scaleLinear()
  .domain([0, 100])    // data range
  .range([0, 500]);    // pixel range

xScale(50);    // → 250 (tengah)

// Ordinal Scale (kategori)
const colorScale = d3.scaleOrdinal()
  .domain(['A', 'B', 'C'])
  .range(['#3b82f6', '#ef4444', '#22c55e']);

colorScale('B');    // → '#ef4444'

// Band Scale (bar chart)
const bandScale = d3.scaleBand()
  .domain(['Jan', 'Feb', 'Mar', 'Apr'])
  .range([0, 400])
  .padding(0.2);    // spacing antar bar

bandScale('Jan');    // → 0 (start position)
bandScale.bandwidth();    // → width of each bar

// Time Scale
const timeScale = d3.scaleTime()
  .domain([new Date('2024-01-01'), new Date('2024-12-31')])
  .range([0, 600]);

// Other scales: scaleSqrt, scaleLog, scalePow, scalePoint
```

Axis

```
// Bikin axis dari scale
const xAxis = d3.axisBottom(xScale);
const yAxis = d3.axisLeft(yScale);

// Render
svg.append('g')
  .attr('transform', `translate(0, ${height})`)
  .call(xAxis);

svg.append('g')
```



```

        .call(yAxis);

// Styling axis
svg.append('g')
  .call(d3.axisBottom(xScale).ticks(5).tickFormat(d => `Rp ${d}rb`));

// Grid lines
svg.append('g')
  .call(d3.axisLeft(yScale)
    .tickSize(-width) // horizontal grid lines
    .tickFormat('') // hide labels
  )
  .selectAll('line')
  .attr('stroke', '#e2e8f0');

```

Bar Chart from Scratch

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>D3 Bar Chart</title>
  <script src="https://d3js.org/d3.v7.min.js"></script>
  <style>
    body { font-family: system-ui, sans-serif; padding: 20px; }
    .chart-container { max-width: 700px; }
    .bar:hover { opacity: 0.7; }
    .tooltip {
      position: absolute; background: #1e293b; color: white;
      padding: 6px 10px; border-radius: 6px; font-size: 13px;
      pointer-events: none; opacity: 0; transition: opacity 0.15s;
    }
  </style>
</head>
<body>
  <h1> D3 Bar Chart</h1>
  <div class="chart-container">
    <div id="chart" style="position: relative;"></div>
  </div>

  <script>
    const data = [
      { kategori: 'Elektronik', nilai: 45 },
      { kategori: 'Fashion', nilai: 30 },

```

```

    { kategori: 'Makanan', nilai: 55 },
    { kategori: 'Buku', nilai: 20 },
    { kategori: 'Olahraga', nilai: 35 },
    { kategori: 'Kesehatan', nilai: 25 },
  ];

  const margin = { top: 20, right: 20, bottom: 40, left: 50 };
  const width = 600 - margin.left - margin.right;
  const height = 400 - margin.top - margin.bottom;

  const svg = d3.select('#chart')
    .append('svg')
    .attr('width', width + margin.left + margin.right)
    .attr('height', height + margin.top + margin.bottom)
    .append('g')
    .attr('transform', `translate(${margin.left},${margin.top})`);

  // Scale
  const xScale = d3.scaleBand()
    .domain(data.map(d => d.kategori))
    .range([0, width])
    .padding(0.2);

  const yScale = d3.scaleLinear()
    .domain([0, d3.max(data, d => d.nilai) + 10])
    .range([height, 0]);

  // Color
  const color = d3.scaleOrdinal(d3.schemeCategory10);

  // Bars
  svg.selectAll('.bar')
    .data(data)
    .enter()
    .append('rect')
    .attr('class', 'bar')
    .attr('x', d => xScale(d.kategori))
    .attr('y', d => yScale(d.nilai))
    .attr('width', xScale.bandwidth())
    .attr('height', d => height - yScale(d.nilai))
    .attr('fill', (d, i) => color(i))
    .attr('rx', 4)
    .on('mouseover', function(event, d) {
      d3.select(this).attr('fill', '#3b82f6');
      tooltip.style('opacity', 1)
        .html(`<strong>${d.kategori}</strong>: ${d.nilai}jt`);
    });

```

```

        .style('left', (event.pageX + 10) + 'px')
        .style('top', (event.pageY - 30) + 'px');
    })
    .on('mouseout', function(event, d) {
        d3.select(this).attr('fill', (_, i) => color(i));
        tooltip.style('opacity', 0);
    });

    // Axis
    svg.append('g')
        .attr('transform', `translate(0,${height})`)
        .call(d3.axisBottom(xScale));

    svg.append('g')
        .call(d3.axisLeft(yScale));

    // Title
    svg.append('text')
        .attr('x', width / 2)
        .attr('y', -5)
        .attr('text-anchor', 'middle')
        .style('font-size', '16px')
        .style('font-weight', 'bold')
        .text('Penjualan per Kategori (jt)');

    // Tooltip
    const tooltip = d3.select('body').append('div')
        .attr('class', 'tooltip')
        .style('position', 'absolute')
        .style('background', '#1e293b')
        .style('color', 'white')
        .style('padding', '6px 10px')
        .style('border-radius', '6px')
        .style('font-size', '13px')
        .style('pointer-events', 'none')
        .style('opacity', 0);
</script>
</body>
</html>

```

Line Chart with Axes

```

<canvas id="lineChart"></canvas>

<script>

```

```

const data = [
  { date: '2024-01', value: 30 },
  { date: '2024-02', value: 45 },
  { date: '2024-03', value: 38 },
  { date: '2024-04', value: 52 },
  { date: '2024-05', value: 48 },
  { date: '2024-06', value: 62 },
];

const margin = { top: 20, right: 20, bottom: 50, left: 50 };
const width = 600 - margin.left - margin.right;
const height = 350 - margin.top - margin.bottom;

const svg = d3.select('#lineChart')
  .append('svg')
  .attr('width', width + margin.left + margin.right)
  .attr('height', height + margin.top + margin.bottom)
  .append('g')
  .attr('transform', `translate(${margin.left},${margin.top})`);

const xScale = d3.scalePoint()
  .domain(data.map(d => d.date))
  .range([0, width]);

const yScale = d3.scaleLinear()
  .domain([0, d3.max(data, d => d.value) + 10])
  .range([height, 0]);

// Line generator
const line = d3.line()
  .x(d => xScale(d.date))
  .y(d => yScale(d.value))
  .curve(d3.curveCatmullRom); // smooth curve

// Area generator
const area = d3.area()
  .x(d => xScale(d.date))
  .y0(height)
  .y1(d => yScale(d.value))
  .curve(d3.curveCatmullRom);

// Draw area
svg.append('path')
  .datum(data)
  .attr('d', area)
  .attr('fill', 'rgba(59, 130, 246, 0.1)');

```

```

// Draw line
svg.append('path')
  .datum(data)
  .attr('d', line)
  .attr('fill', 'none')
  .attr('stroke', '#3b82f6')
  .attr('stroke-width', 3);

// Points
svg.selectAll('.point')
  .data(data)
  .enter()
  .append('circle')
  .attr('class', 'point')
  .attr('cx', d => xScale(d.date))
  .attr('cy', d => yScale(d.value))
  .attr('r', 5)
  .attr('fill', 'white')
  .attr('stroke', '#3b82f6')
  .attr('stroke-width', 3);

// Labels on points
svg.selectAll('.label')
  .data(data)
  .enter()
  .append('text')
  .attr('x', d => xScale(d.date))
  .attr('y', d => yScale(d.value) - 12)
  .attr('text-anchor', 'middle')
  .style('font-size', '12px')
  .style('fill', '#333')
  .text(d => d.value);

// Axis
svg.append('g')
  .attr('transform', `translate(0,${height})`)
  .call(d3.axisBottom(xScale));

svg.append('g')
  .call(d3.axisLeft(yScale));

// Grid
svg.append('g')
  .call(d3.axisLeft(yScale)
    .tickSize(-width)

```

```

        .tickFormat('')
    )
    .selectAll('line')
    .attr('stroke', '#e2e8f0')
    .attr('stroke-dasharray', '3 3');
</script>

```

Transition & Animation

```

// Basic transition
d3.select('rect')
  .transition()
  .duration(800)
  .delay(200)
  .ease(d3.easeElastic)
  .attr('width', 200)
  .attr('fill', '#ef4444');

// Staggered transition
svg.selectAll('.bar')
  .data(data)
  .enter()
  .append('rect')
  .attr('y', height)
  .attr('height', 0)
  .transition()
  .delay((d, i) => i * 50)
  .duration(600)
  .ease(d3.easeBackOut)
  .attr('y', d => yScale(d.nilai))
  .attr('height', d => height - yScale(d.nilai));

// Update transition (data change)
function updateChart(newData) {
  const bars = svg.selectAll('.bar').data(newData, d => d.kategori);

  // EXIT
  bars.exit()
    .transition()
    .duration(400)
    .attr('y', height)
    .attr('height', 0)
    .remove();

  // ENTER

```

```

const enter = bars.enter()
  .append('rect')
  .attr('class', 'bar')
  .attr('y', height)
  .attr('height', 0);

// MERGE + UPDATE
enter.merge(bars)
  .transition()
  .duration(600)
  .delay((d, i) => i * 30)
  .attr('x', d => xScale(d.kategori))
  .attr('y', d => yScale(d.nilai))
  .attr('width', xScale.bandwidth())
  .attr('height', d => height - yScale(d.nilai))
  .attr('fill', (d, i) => color(i));
}

// Tween - interpolate nilai
d3.select('text')
  .transition()
  .duration(1000)
  .tween('text', function() {
    const that = this;
    const current = 0;
    const target = 100;
    const interpolator = d3.interpolate(current, target);
    return (t) => {
      that.textContent = Math.round(interpolator(t));
    };
  });

```

Easing Functions

```

d3.easeLinear           // konstan
d3.easeQuad            // quadratic in/out
d3.easeCubic           // cubic in/out
d3.easeElastic         // elastic bounce (karet)
d3.easeBounce          // bounce di akhir
d3.easeBackOut         // overshoot sedikit
d3.easeCircle          // circular in/out

```

Latihan

1. **Bar Chart Interaktif:** Data: populasi 5 kota besar Indonesia (Jakarta 10.6jt, Surabaya 2.97jt, Bandung 2.44jt, Medan 2.11jt, Semarang 1.63jt). Bikin bar chart pake D3 dengan: (a) color scale categorical, (b) tooltip nunjukin nama kota + populasi, (c) bar muncul dengan animasi stagger dari bawah. Horizontal bar chart.
2. **Multi-Line Chart:** Data: suhu harian 3 kota (Jakarta, Bandung, Surabaya) selama 7 hari. Bikin line chart dengan 3 line (warna berbeda), legend di kanan atas, grid lines horizontal, dan titik data dengan label suhu. Pake `d3.scaleTime()` untuk sumbu x.
3. **Scatter Plot dengan Transisi:** Generate 50 data point random (x: 0-100, y: 0-100, group: A/B/C). Bikin scatter plot dengan: (a) warna per group, (b) radius proportional ke nilai ke-3 (z), (c) tooltip, (d) tombol "Acak Ulang" yang update data dengan transisi. Pake general update pattern.
4. **Animated Pie Chart:** Data: budget bulanan (makan 2.5jt, transport 1.2jt, hiburan 800rb, tagihan 1.5jt, tabungan 1jt). Bikin pie/donut chart dari scratch pake D3 (pake `d3.arc()` dan `d3.pie()`). Tambah: (a) animasi masuk (arc grow dari 0), (b) hover effect (arc expand sedikit), (c) label persentase di tiap slice.

33.4 Dashboard Real-World: Real-Time Data, Layout, Dark Mode & Filtering

Real-Time Data dengan WebSocket + Chart.js

WebSocket Server (Node.js)

```
npm init -y
npm install ws express

// server.js - WebSocket + Express
const express = require('express');
const http = require('http');
const { WebSocketServer } = require('ws');

const app = express();
app.use(express.static('public'));

const server = http.createServer(app);
const wss = new WebSocketServer({ server });
```



```

// Generate market data
function generatePrice(base = 50000) {
  const change = (Math.random() - 0.5) * 1000;
  return Math.round((base + change) * 100) / 100;
}

function generateData() {
  return {
    timestamp: new Date().toISOString(),
    bitcoin: generatePrice(650000000),
    ethereum: generatePrice(350000000),
    solana: generatePrice(150000),
    volume: Math.floor(Math.random() * 1000) + 500,
  };
}

wss.on('connection', (ws) => {
  console.log('Client connected');

  const interval = setInterval(() => {
    if (ws.readyState === ws.OPEN) {
      ws.send(JSON.stringify(generateData()));
    }
  }, 2000); // every 2 seconds

  ws.on('close', () => {
    clearInterval(interval);
    console.log('Client disconnected');
  });
});

server.listen(3000, () => {
  console.log('🚀 Server running at http://localhost:3000');
});

```

WebSocket Client + Chart.js

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Real-Time Dashboard</title>
  <script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.1/dist/chart.umd.min.js">

```

```

<script src="https://cdn.jsdelivr.net/npm/chartjs-adapter-date-fns@3/dist/chartjs-adapter-date-fns.bundle.min.js"></script>
<style>
  * { margin: 0; padding: 0; box-sizing: border-box; }
  body { font-family: system-ui, sans-serif; background: #0f172a; color: #e2e8f0; }
  h1 { margin-bottom: 20px; font-size: 24px; }
  .stats { display: flex; gap: 12px; margin-bottom: 20px; flex-wrap: wrap; }
  .card {
    background: #1e293b; border-radius: 12px; padding: 16px 20px;
    min-width: 160px; flex: 1;
  }
  .card .label { font-size: 12px; color: #94a3b8; text-transform: uppercase; }
  .card .value { font-size: 24px; font-weight: 700; margin-top: 4px; }
  .card .change { font-size: 13px; margin-top: 2px; }
  .up { color: #22c55e; }
  .down { color: #ef4444; }
  .chart-container { background: #1e293b; border-radius: 12px; padding: 20px; }
  canvas { max-height: 350px; }
</style>
</head>
<body>
  <h1>📊 Real-Time Crypto Dashboard</h1>

  <div class="stats" id="stats">
    <div class="card">
      <div class="label">Bitcoin</div>
      <div class="value" id="btc-price"></div>
      <div class="change" id="btc-change">Menunggu data...</div>
    </div>
    <div class="card">
      <div class="label">Ethereum</div>
      <div class="value" id="eth-price"></div>
      <div class="change" id="eth-change">Menunggu data...</div>
    </div>
    <div class="card">
      <div class="label">Solana</div>
      <div class="value" id="sol-price"></div>
      <div class="change" id="sol-change">Menunggu data...</div>
    </div>
    <div class="card">
      <div class="label">Volume (24h)</div>
      <div class="value" id="vol-price"></div>
    </div>
  </div>

  <div class="chart-container">
    <canvas id="realtimeChart"></canvas>
  </div>

```

```

</div>

<script>
  // Setup chart
  const ctx = document.getElementById('realtimeChart').getContext('2d');
  const maxPoints = 50;

  const chart = new Chart(ctx, {
    type: 'line',
    data: {
      datasets: [
        { label: 'Bitcoin', data: [], borderColor: '#f7931a', pointRadius: 0, },
        { label: 'Ethereum', data: [], borderColor: '#8b5cf6', pointRadius: 0, },
        { label: 'Solana', data: [], borderColor: '#22c55e', pointRadius: 0, },
      ],
    },
    options: {
      responsive: true,
      animation: { duration: 300 },
      scales: {
        x: {
          type: 'time',
          time: { unit: 'second', displayFormats: { second: 'HH:mm:ss' } },
          grid: { color: 'rgba(255,255,255,0.05)' },
          ticks: { color: '#94a3b8' },
        },
        y: {
          grid: { color: 'rgba(255,255,255,0.05)' },
          ticks: { color: '#94a3b8' },
        },
      },
      plugins: {
        legend: { labels: { color: '#e2e8f0' } },
      },
    },
  });

  // Track previous prices for change %
  let prevPrices = { bitcoin: null, ethereum: null, solana: null };

  // Connect WebSocket
  const ws = new WebSocket('ws://localhost:3000');

  ws.onmessage = (event) => {
    const data = JSON.parse(event.data);
    const time = new Date(data.timestamp);
  }

```

```

    const formatted = time.toLocaleTimeString('id-ID');

    // Update stats
    document.getElementById('btc-price').textContent =
      `Rp ${data.bitcoin.toLocaleString('id-ID')}`;
    document.getElementById('eth-price').textContent =
      `Rp ${data.ethereum.toLocaleString('id-ID')}`;
    document.getElementById('sol-price').textContent =
      `Rp ${data.solana.toLocaleString('id-ID')}`;
    document.getElementById('vol-price').textContent =
      `${data.volume.toLocaleString('id-ID')} BTC`;

    // Change %
    for (const [key, label] of [['bitcoin', 'btc'], ['ethereum', 'eth'], ['solana', 'sol']]) {
      const prev = prevPrices[key];
      const current = data[key];
      if (prev !== null) {
        const change = ((current - prev) / prev * 100).toFixed(2);
        const el = document.getElementById(`${label}-change`);
        el.textContent = `${change > 0 ? '+' : ''}${change}%`;
        el.className = `change ${change >= 0 ? 'up' : 'down'}`;
      }
      prevPrices[key] = current;
    }

    // Push data to chart
    chart.data.datasets[0].data.push({ x: time, y: data.bitcoin });
    chart.data.datasets[1].data.push({ x: time, y: data.ethereum });
    chart.data.datasets[2].data.push({ x: time, y: data.solana });

    // Keep max points
    if (chart.data.datasets[0].data.length > maxPoints) {
      chart.data.datasets.forEach(ds => ds.data.shift());
    }

    chart.update('none'); // 'none' = skip animation for perf
  };
</script>
</body>
</html>

```

Dashboard Layout dengan CSS Grid

```

<!DOCTYPE html>
<html lang="en">

```

```

<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Dashboard Layout</title>
  <script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.1/dist/chart.umd.min.js"></script>
  <style>
    * { margin: 0; padding: 0; box-sizing: border-box; }

    :root {
      --bg: #f8fafc;
      --surface: #ffffff;
      --text: #0f172a;
      --text-secondary: #64748b;
      --border: #e2e8f0;
      --accent: #3b82f6;
    }

    body {
      font-family: system-ui, sans-serif;
      background: var(--bg);
      color: var(--text);
      min-height: 100vh;
    }

    /* HEADER */
    .header {
      background: var(--surface);
      border-bottom: 1px solid var(--border);
      padding: 16px 24px;
      display: flex;
      align-items: center;
      justify-content: space-between;
    }
    .header h1 { font-size: 20px; display: flex; align-items: center; gap: 8px; }
    .header-controls { display: flex; gap: 8px; align-items: center; }

    /* SIDEBAR */
    .layout {
      display: grid;
      grid-template-columns: 220px 1fr;
      min-height: calc(100vh - 64px);
    }
    .sidebar {
      background: var(--surface);
      border-right: 1px solid var(--border);
      padding: 20px;

```

```

}
.sidebar h3 { font-size: 12px; text-transform: uppercase; color: var(--text-
.sidebar nav { display: flex; flex-direction: column; gap: 4px; }
.sidebar a {
  padding: 8px 12px; border-radius: 8px; text-decoration: none;
  color: var(--text); font-size: 14px; transition: background 0.15s;
}
.sidebar a:hover { background: #e2e8f0; }
.sidebar a.active { background: var(--accent); color: white; }

/* GRID CHARTS */
.main { padding: 20px; }
.chart-grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 16px;
  margin-bottom: 20px;
}
.chart-card {
  background: var(--surface);
  border-radius: 12px;
  padding: 20px;
  box-shadow: 0 1px 3px rgba(0,0,0,0.06);
}
.chart-card h2 {
  font-size: 14px;
  color: var(--text-secondary);
  margin-bottom: 12px;
  font-weight: 600;
}
.chart-card canvas { max-height: 250px; }

/* FULL WIDTH */
.chart-full {
  grid-column: 1 / -1;
}

/* FILTER BAR */
.filter-bar {
  background: var(--surface);
  border-radius: 12px;
  padding: 16px 20px;
  margin-bottom: 20px;
  display: flex;
  gap: 12px;
  align-items: center;

```

```

    flex-wrap: wrap;
    box-shadow: 0 1px 3px rgba(0,0,0,0.06);
  }
  .filter-bar label { font-size: 13px; font-weight: 500; }
  .filter-bar select, .filter-bar input {
    padding: 6px 12px;
    border: 1px solid var(--border);
    border-radius: 6px;
    font-size: 13px;
    background: var(--bg);
    color: var(--text);
  }
  .filter-bar button {
    padding: 6px 16px;
    background: var(--accent);
    color: white;
    border: none;
    border-radius: 6px;
    cursor: pointer;
    font-size: 13px;
  }
}

@media (max-width: 768px) {
  .layout { grid-template-columns: 1fr; }
  .sidebar { display: none; }
  .chart-grid { grid-template-columns: 1fr; }
}
</style>
</head>
<body>
  <header class="header">
    <h1> Analytics Dashboard</h1>
    <div class="header-controls">
      <span id="lastUpdate" style="font-size:13px;color:var(--text-secondary);">
    </div>
  </header>

  <div class="layout">
    <aside class="sidebar">
      <h3>Menu</h3>
      <nav>
        <a href="#" class="active">Overview</a>
        <a href="#">Revenue</a>
        <a href="#">Users</a>
        <a href="#">Traffic</a>
        <a href="#">Settings</a>
      </nav>
    </aside>
  </div>

```

```

    </nav>
</aside>

<main class="main">
  <!-- Filter Bar -->
  <div class="filter-bar">
    <label>Periode:</label>
    <select id="periodSelect">
      <option value="7">7 Hari</option>
      <option value="30" selected>30 Hari</option>
      <option value="90">90 Hari</option>
    </select>
    <label>Kategori:</label>
    <select id="categorySelect">
      <option value="all">Semua</option>
      <option value="produk-a">Produk A</option>
      <option value="produk-b">Produk B</option>
      <option value="produk-c">Produk C</option>
    </select>
    <label>Dari:</label>
    <input type="date" id="dateFrom" />
    <label>Sampai:</label>
    <input type="date" id="dateTo" />
    <button onclick="applyFilter()">Terapkan</button>
  </div>

  <!-- Chart Grid -->
  <div class="chart-grid">
    <div class="chart-card chart-full">
      <h2>📊 Revenue Overview</h2>
      <canvas id="revenueChart"></canvas>
    </div>
    <div class="chart-card">
      <h2>📊 User Growth</h2>
      <canvas id="userChart"></canvas>
    </div>
    <div class="chart-card">
      <h2>📊 Traffic Sources</h2>
      <canvas id="trafficChart"></canvas>
    </div>
    <div class="chart-card">
      <h2>📊 Top Products</h2>
      <canvas id="productChart"></canvas>
    </div>
    <div class="chart-card">
      <h2>📊 Regional</h2>

```



```

        <canvas id="regionChart"></canvas>
      </div>
    </div>
  </main>
</div>

<script>
  // Sample data
  const labels = Array.from({ length: 30 }, (_, i) => `Hari ${i + 1}`);
  const revenueData = labels.map(() => Math.floor(Math.random() * 80) + 20);
  const userData = labels.map(_, i) => Math.floor(100 + i * 5 + Math.random());
  const trafficData = [35, 25, 20, 12, 8]; // Organic, Direct, Social, Referral
  const productData = [42, 28, 18, 12];
  const regionData = [35, 25, 20, 12, 8];

  // Revenue Chart
  new Chart(document.getElementById('revenueChart'), {
    type: 'line',
    data: {
      labels,
      datasets: [{
        label: 'Revenue (jt)',
        data: revenueData,
        borderColor: '#3b82f6',
        backgroundColor: 'rgba(59,130,246,0.1)',
        fill: true,
        tension: 0.3,
      }],
    },
    options: { responsive: true, plugins: { legend: { display: false } } },
  });

  // User Growth
  new Chart(document.getElementById('userChart'), {
    type: 'bar',
    data: {
      labels: labels.filter(_, i) => i % 5 === 0,
      datasets: [{
        label: 'Users',
        data: userData.filter(_, i) => i % 5 === 0,
        backgroundColor: '#22c55e',
      }],
    },
    options: { responsive: true, plugins: { legend: { display: false } } },
  });

```

```

// Traffic Sources (Pie)
new Chart(document.getElementById('trafficChart'), {
  type: 'doughnut',
  data: {
    labels: ['Organic Search', 'Direct', 'Social Media', 'Referral', 'Other'],
    datasets: [{
      data: trafficData,
      backgroundColor: ['#3b82f6', '#22c55e', '#f59e0b', '#ef4444', '#8b5cf6'],
    }],
  },
  options: { responsive: true, plugins: { legend: { position: 'bottom' } } }
});

// Top Products (Horizontal Bar)
new Chart(document.getElementById('productChart'), {
  type: 'bar',
  data: {
    labels: ['Produk A', 'Produk B', 'Produk C', 'Produk D'],
    datasets: [{
      label: 'Penjualan',
      data: productData,
      backgroundColor: ['#3b82f6', '#ef4444', '#22c55e', '#f59e0b'],
    }],
  },
  options: {
    responsive: true,
    indexAxis: 'y',
    plugins: { legend: { display: false } } },
});

// Regional
new Chart(document.getElementById('regionChart'), {
  type: 'bar',
  data: {
    labels: ['Jakarta', 'Bandung', 'Surabaya', 'Medan', 'Yogyakarta'],
    datasets: [{
      label: 'Penjualan (jt)',
      data: regionData,
      backgroundColor: '#8b5cf6',
    }],
  },
  options: { responsive: true, plugins: { legend: { display: false } } }
});

function applyFilter() {

```

```

    const period = document.getElementById('periodSelect').value;
    const category = document.getElementById('categorySelect').value;
    const dateFrom = document.getElementById('dateFrom').value;
    const dateTo = document.getElementById('dateTo').value;
    console.log('Filter:', { period, category, dateFrom, dateTo });
    alert('Filter applied! (check console)');
  }

  // Update timestamp
  document.getElementById('lastUpdate').textContent = `Terakhir: ${new Date()}.
</script>
</body>
</html>

```

Dark Mode Dashboard

```

/* Dark mode – toggle class di body */
:root {
  /* Light */
  --bg: #f8fafc;
  --surface: #ffffff;
  --text: #0f172a;
  --text-secondary: #64748b;
  --border: #e2e8f0;
}

body.dark {
  --bg: #0f172a;
  --surface: #1e293b;
  --text: #e2e8f0;
  --text-secondary: #94a3b8;
  --border: #334155;
}

/* Toggle button */
.theme-toggle {
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 8px;
  padding: 6px 12px;
  cursor: pointer;
  color: var(--text);
  font-size: 14px;
}

```

```

/* Chart.js dark mode – update options */
function getChartOptions(dark) {
  return {
    scales: {
      x: {
        grid: { color: dark ? 'rgba(255,255,255,0.05)' : 'rgba(0,0,0,0.05)' },
        ticks: { color: dark ? '#94a3b8' : '#64748b' },
      },
      y: {
        grid: { color: dark ? 'rgba(255,255,255,0.05)' : 'rgba(0,0,0,0.05)' },
        ticks: { color: dark ? '#94a3b8' : '#64748b' },
      },
    },
    plugins: {
      legend: { labels: { color: dark ? '#e2e8f0' : '#0f172a' } },
    },
  };
}

// Toggle theme
const toggleBtn = document.getElementById('themeToggle');
toggleBtn.addEventListener('click', () => {
  document.body.classList.toggle('dark');
  const isDark = document.body.classList.contains('dark');
  toggleBtn.textContent = isDark ? '* Light' : '□ Dark';

  // Update semua chart
  charts.forEach(chart => {
    chart.options = { ...chart.options, ...getChartOptions(isDark) };
    chart.update();
  });
});

```

Filtering & Date Range

```

// Data source – lengkap
const allData = [
  { date: '2024-01-05', category: 'produk-a', value: 30 },
  { date: '2024-01-12', category: 'produk-b', value: 45 },
  // ... banyak data
];

function filterData(data, { category, dateFrom, dateTo }) {
  return data.filter(d => {
    if (category !== 'all' && d.category !== category) return false;
    if (dateFrom && new Date(d.date) < new Date(dateFrom)) return false;
  });
}

```

```

        if (dateTo && new Date(d.date) > new Date(dateTo)) return false;
        return true;
    });
}

function updateCharts(filteredData) {
    // Aggregate by date
    const aggregated = {};
    filteredData.forEach(d => {
        aggregated[d.date] = (aggregated[d.date] || 0) + d.value;
    });

    const labels = Object.keys(aggregated).sort();
    const values = labels.map(l => aggregated[l]);

    // Update chart data
    revenueChart.data.labels = labels;
    revenueChart.data.datasets[0].data = values;
    revenueChart.update();
}

```

Multiple Chart Sync

Sinkronisasi hover/zoom antar chart.

```

// Sync tooltip crosshair antar chart
function syncCrosshair(sourceChart, targetChart) {
    sourceChart.canvas.addEventListener('mousemove', (e) => {
        const rect = sourceChart.canvas.getBoundingClientRect();
        const x = (e.clientX - rect.left) / rect.width;

        // Hitung index di source
        const meta = sourceChart.getDatasetMeta(0);
        const index = Math.round(x * (meta.data.length - 1));
        if (index < 0 || index >= meta.data.length) return;

        // Set active elements di semua chart
        [sourceChart, targetChart].forEach(chart => {
            chart.setActiveElements([
                { dataIndex: 0, index: Math.min(index, chart.data.labels.length - 1)
            });
            chart.draw();
        });
    });
}

```

```
// Sync zoom level antar time series
function syncZoom(sourceChart, targetCharts) {
  sourceChart.canvas.addEventListener('mouseup', () => {
    const xScale = sourceChart.scales.x;
    const { min, max } = xScale;

    targetCharts.forEach(chart => {
      chart.options.scales.x.min = min;
      chart.options.scales.x.max = max;
      chart.update();
    });
  });
}
```

Latihan

1. **Live CPU Monitor Dashboard:** Bikin dashboard real-time dengan WebSocket server (simulasi CPU/memory/disk usage — update tiap 1 detik). Tampilkan 3 line chart (CPU%, Memory%, Disk%) + stat cards (current value, min, max, avg). Pake dark theme. Data retention: last 60 detik. Tambah tombol pause/resume.
2. **E-Commerce Dashboard:** Bikin dashboard 6 chart: (1) revenue line chart (7 hari), (2) orders bar chart per hari, (3) pie chart kategori produk, (4) horizontal bar top 5 produk, (5) area chart conversion rate, (6) scatter plot qty vs revenue. Layout 3 kolom × 2 baris pake CSS Grid. Tambah filter tanggal & kategori. Responsive — jadi 1 kolom di mobile.
3. **Dark Mode + Theme Switcher:** Ambil dashboard dari latihan 2. Tambah: (a) toggle dark/light mode, (b) chart colors auto-adjust, (c) persist theme preference di localStorage, (d) smooth transition CSS antar mode. Wajib update chart options pas toggle (scale grid colors, text colors).
4. **Synchronized Multi-Chart + Export:** Bikin 3 chart yang saling sync: (a) line chart overview (full range), (b) bar chart zoom detail (sync selection dari overview), (c) pie chart distribusi berdasarkan range yang dipilih. Tambah: (a) sync hover crosshair antar chart, (b) klik di chart overview set range chart detail, (c) export all charts sebagai 1 image (pake html2canvas atau manual canvas composite).